

الفنية الشاملة والتفصيلية لمشروع منصة حجز الطيران YBF الوثيقة

دليل هندسة النظام، الشيفرة البرمجية، قواعد البيانات، والتحديثات التطويرية الأخيرة

اسم المشروع:	منصة YBF لحجز تذاكر الطيران
لغة الواجهة الأمامية:	React.js + Vite + Tailwind CSS
الواجهة الخلفية:	Node.js (Express) + Prisma ORM
قاعدة البيانات:	MySQL Database
إصدار الوثيقة:	(أول إصدار) 2.0
تاريخ الإصدار:	2026 ويولي

1. البنية الهندسية وتصميم النظام العام

(Client-Server) لضمان استقرار العمليات البرمجية وفصل منطق الأعمال والواجهات التفاعلية عن الاستعلامات وإدارة الموارد المباشرة في قاعدة البيانات. ومطار سينون) بالمطارات والعواصم الدولية الأكثر طلبا (مثل القاهرة، الرياض، جدة، دبي، وعمان). يعتمد النظام على بنية العميل والخادم (Architecture) تم تصميم وتطوير منصة YBF كمنصة رقمية موحدة تخدم قطاع الطيران اليمني، بهدف ربط المطارات المحلية (مثل مطار عدن الدولي، مطار الريان بالمكلا،

دورة حياة الحجز والخطوات المتبعة:

وتاريخ المغادرة والعودة، مع تحديد عدد الركاب. يتم مزامنة هذه البيانات عبر سياق البحث (SearchContext) لضمان الاحتفاظ بها خلال تنقله بالموقع. البحث وتحديد المعايير: يدخل العميل إلى الصفحة الرئيسية ويقوم بتحديد نوع الرحلة (مسار واحد، مسارين، أو مدن متعددة) واختيار مطار الإقلاع والوصول

1.

المتوفرة. تتيح الصفحة تصفية النتائج بحسب الخطوط الجوية المفضلة (اليمنية، بلقيس، فلاي عدن) أو ترتيبها تصاعديا وتنزليا بحسب الأسعار ومدة الرحلة.

2. استعراض ومقارنة الرحلات: يتم توجيه المستخدم لصفحة البحث (Search.jsx) التي تقوم تلقائيا بطلب البيانات من السيرفر وعرض الرحلات

التي تعرض نموذج طائرة تفاعلي حقيقي. يتيح النظام للعميل تمييز درجات السفر (أعمال أو سياحية) وحجز المقعد مؤقتا عبر آلية قفل المقاعد المبرمجة.

3. حجز المقعد التفاعلي (Interactive Seat Map): بعد اختيار الرحلة المناسبة، يتم نقل العميل لصفحة المقاعد (Seats.jsx)

يتم ملء بيانات الركاب (الاسم الكامل، رقم جواز السفر وتاريخ انتهائه، وتاريخ الميلاد) ثم حفظها وتدقيقها للتأكد من مطابقتها لشروط الطيران الدولي.

4. إدخال بيانات المسافرين والتحقق:

دفع يدويا لمراجعة الإدارة. فور تأكيد الدفع، يتم تغيير حالة الحجز لـ 'مؤكد' وإنشاء تذكرة إلكترونية PDF تحتوي على رمز استجابة سريعة QR فريد.

5. بوابة الدفع الآمنة وإصدار التذاكر: يتم توجيه العميل للدفع إما عبر Stripe لبطاقات الائتمان أو عن طريق رفع إيصال

2. هندسة الواجهة الأمامية (Frontend Detail)

باستخدام مكتبة React مع أداة البناء الحديثة Vite لسرعة فائقة في معالجة الملفات الساكنة وتحسين التحميل المتوازي للملفات. وقد قمنا بهيكلتها بالشكل التالي:
تم بناء الواجهة الأمامية بالكامل

المكونات البرمجية الأساسية (Components):

- HeroSearchPanel.jsx: تبديل محطة الإقلاع والوصول والتحكم بعدد الركاب وحقول التواريخ المخصصة. يتميز بدعم تفاعلي متقدم للهواتف الذكية: هذا الملف يمثل قلب لوحة البحث في الصفحة الرئيسية. يتحكم بتبديل الواجهات الفرعية لأنواع الرحلات ويحتوي على منطق للمسافر (البحث □ المقاعد □ الركاب □ الدفع □ التأكيد) وعرض حالة التقدم بشكل رسومي أنيق يعزز تجربة المستخدم
- BookingStepper.jsx: شريط الخطوات المتواجد في أعلى صفحات الحجز والبحث. يقوم بتعريف المسار الحالي
- ScrollReveal.jsx: لمسة جمالية راقية ويزيد تفاعلية الواجهات (Transitions and Animations) مكون مخصص لتطبيق تأثيرات حركية عند تمرير الصفحة لأسفل، مما يضيف

صفحات النظام الرئيسية (Pages):

- Home.jsx: خدمات المنصة، الإحصائيات، قسم الأسئلة الشائعة تفاعليا، بالإضافة للقسم الشهير "الوجهات الأكثر طلبا" (القاهرة، الرياض، دبي، عمان: تحتوي على عرض
- Search.jsx: وتتضمن شريط فلترة جانبي لشركات الطيران ومحاكاة ذكية للفرز API صفحة محرك البحث المتقدم. تقوم بجدولة الرحلات والاتصال بال: لطلب الرحلات المناسبة،
- Seats.jsx: المقاعد المؤقت لمنع تصادم الحجوزات، مع واجهة ثلاثية الأبعاد تحاكي توزيع المقاعد API تحتوي على واجهة اختيار المقاعد والربط مع حجز.
- AdminDashboard.jsx: لوحة تحكم الإدارة الشاملة للتحكم بالمستخدمين والشركات وتتبع الأرباح وإدارة نظام الشات والدعم الفني وإعدادات Markup وتعديل أسعار الصرف
- CompanyDashboard.jsx: شريكة لإدخال رحلاتها الجديدة، تتبع حجوزات الركاب على طائراتها، واستعراض التقارير المالية الخاصة بها فقط: لوحة مخصصة لكل شركة طيران

3. هندسة الواجهة الخلفية وقاعدة البيانات (Backend & Database)

مسارات البيانات وتنسيق الاتصالات. يتم تنظيم وحماية البيانات بواسطة قاعدة بيانات MySQL العلاقية مع Prisma ORM لتوليد الجداول وإدارتها ببسر. تعتمد الواجهة الخلفية على Node.js وإطار Express.js لإدارة

هيكلية قاعدة البيانات بالتفصيل:

الوظيفة التشغيلية في النظام	الحقول والمفاتيح الأساسية والعلاقات	اسم الجدول
تخزين بيانات حسابات المسافرين وتأمين كلمات مرورهم المشفرة.	users (PK) email, phone, full_name, password	users
تحديد صلاحيات المشرفين (ادمين عام أو ممثل شركة طيران محددة).	admins (PK) username, password, role, airline_code	admins
حفظ بيانات شركات الطيران الشريكة وربط الرحلات بها.	airlines (PK) company_name, airline_code	airlines
تفاصيل الرحلات، مطارات الإقلاع والوصول، المقاعد الكلية والشاغرة.	flights (PK) flight_number, price, departure_time	flights
حفظ سجلات الحجوزات مع حالات الدفع والرموز الخاصة بكل حجز.	bookings (PK) user_id (FK), flight_id (FK), status	bookings
حفظ معلومات جوازات السفر الخاصة بالمسافرين وتديق صلاحياتها.	passengers (PK) passport_number, nationality, gender	passengers
التحكم بحجز مقاعد الطائرات وتفعيل قفل المقاعد المؤقت أثناء الشراء.	seats (PK) flight_id (FK), seat_number, status, hold_until	seats
أرشيف رسائل المحادثات المباشرة بين الدعم الفني والزوار.	chat_sessions (PK) session_id (FK), sender, message, created_at	chat_sessions

العمليات البرمجية الدقيقة في السيرفر (server.js):

- لتهيئة بوابة دفع رسمية. يتم تمرير رقم الحجز وقيمة التذاكر كميّاتنا (Metadata) لـ Stripe. بعد نجاح العملية، يقوم السيرفر بتأكيد حالة الحجز تلقائياً.
- معالجة المدفوعات الأمانة: عند النقر على الدفع، يتصل مسار `api/create-checkout-session/` بـ Stripe API لتصميم HTML منسق لبطاقة الصعود وتذكرة المسافر، ثم يلتقط لقطة للمستند ويولد تذكرة PDF تحتوي على رمز QR ذو تشفير فريد للتحقق بالمطار.
- مسار الـ `PDF /api/bookings/:id/ticket`. يستخدم السيرفر مكتبة Puppeteer لتشغيل متصفح خفي (Headless Browser) وتوجيهه لتوليد التذاكر التلقائي. يتم استدعاء

4. شرح الأكواد البرمجية والتحسينات المنجزة بالتفصيل

لقد قمنا مؤخرا بعملية صيانة وتطوير مكثفة لضمان تشغيل النظام بأعلى كفاءة ممكنة، وسنوضح هنا أهم الأكواد والتغييرات التي قمنا بها:

أولاً: إزالة التعليقات وتنظيف الكود (Clean Code):

التعليقات التوضيحية الأحادية السطر والكتل متعددة السطور) وتطبيقه على 44 ملف برمجي بالكامل مع الحفاظ التام على سلامة منطق الأكواد البرمجية. لتسهيل عملية التطوير وضمان نظافة الكود في بيئة الإنتاج، قمنا بتطوير سكربت بايثون مخصص لمعالجة النصوص يقوم بحذف تعليقات الكود البرمجية (مثل

ثانياً: مسار جلب أسعار الوجهات المفضلة ديناميكياً (Dynamic Pricing API):

لتجنب عرض أسعار ثابتة ووهمية على الواجهة الرئيسية للزوار، قمنا بإجراء تعديل مزدوج على السيرفر والواجهة الأمامية:

1. كود السيرفر المضاف في server.js لجمع الأسعار:

```
app.get('/api/flights/starting-prices', async (req, res) => {
  let connection;
  try {
    connection = await mysql.createConnection(getDbConfig());
    const [rows] = await connection.execute(
      "SELECT airportDestination_code AS destination, MIN(price) AS minPrice "
      "FROM flights WHERE status = 'active' AND departure_time >= NOW() "
      "GROUP BY airportDestination_code"
    );
    const prices = {};
    for (const r of rows) {
      prices[r.destination.toUpperCase()] = parseFloat(r.minPrice) || 0;
    }
    res.json({ success: true, prices });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  } finally {
    if (connection) await connection.end();
  }
});
```

2. طريقة تحديث كود React في الصفحة الرئيسية Home.jsx لاستيعاب الأسعار الديناميكية:

```
const [prices, setPrices] = useState({ cairo: 180, dubai: 220, riyadh: 250, amman: 190 });
useEffect(() => {
  const fetchStartingPrices = async () => {
    try {
      const baseUrl = import.meta.env.VITE_API_URL || 'http://localhost:8080';
      const response = await fetch(`${baseUrl}/api/flights/starting-prices`);
      const data = await response.json();
      if (data.success && data.prices) {
        const codeTold = { 'CAI': 'cairo', 'DXB': 'dubai', 'RUH': 'riyadh', 'AMM': 'amman' };
        setPrices(prev => {
          const next = { ...prev };
          Object.entries(data.prices).forEach(([code, price]) => {
            const id = codeTold[code];
            if (id && price > 0) next[id] = price;
          });
          return next;
        });
      }
    }
  };
  fetchStartingPrices();
}, []);
```

```
    });  
  }  
} catch (err) { console.error(err); }  
};  
fetchStartingPrices();  
}, []);
```

5. الفوائد التقنية وتنسيق مجلدات العمل الحالية

أصبحت هيكلية مشروع YBF الآن نظيفة ومثالية تماما للتشغيل الفعلي. إليك توضيح لأهم المجلدات وحالتها بعد الصيانة:

• مجلد receipts (إيصالات الدفع):

برفعها عند الدفع بالتحويل البنكي اليدوي، ولولا بقاء هذا المجلد فارغا لحدث انهيار (Crash) في الخادم عند محاولة رفع أول إيصال دفع حقيقي. قمنا بحذف الصورة التجريبية المؤقتة وتصفية المجلد بالكامل مع وضع ملف gitkeep فيه. هذا المجلد ضروري جدا لتخزين الصور التي يقوم المستخدمون

• مجلد dist (النسخة الجاهزة للإنتاج):

مجلد dist مخرجات أداة Vite النهائية لرفعها واستضافتها على خادم السيرفر، ولا ينصح بتعديل ملفاته إطلاقا بل تعديل الملفات في مجلد src و public. يحتوي هذا المجلد على النسخة التي تم بناؤها وتوليدها تلقائيا عند تشغيل أمر البناء npm run build. يعتبر

• استبدال Favicon (أيقونة المتصفح):

في index.html لضمان التوافق المطلق مع جميع شاشات الموبايل ومتصفحات الإنترنت القديمة والحديثة، مما يعزز الهوية البصرية الرسمية للموقع. تم استبدال أيقونة المتصفح التالفة بصيغة SVG بأيقونة PNG احترافية وجذابة، وتحديث مسار الاستدعاء

توصيات الاستخدام والتطوير المستقبلي:

1. لضمان جودة الأداء وسلاسة الحجزات، يوصى دائما بتشغيل نظام تنظيف دوري للمقاعد المعقدة (Held Seats) التي لم يتم دفع قيمتها خلال 15 دقيقة.
2. في تعديل أو إضافة ميزات جديدة، تأكد من تعديل الملفات في مجلد `src` ثم تشغيل أمر البناء `npm run build` لتحديث مجلد `dist` المرفوع تلقائيا. عند رغبتك
3. يفضل الاحتفاظ بنسخ احتياطية دورية لقاعدة البيانات MySQL لضمان سلامة بيانات المسافرين والرحلات النشطة وتأكد الحجزات بانتظام.

خاتمة الوثيقة:

المفصل ليكون المرجع الأساسي والشامل لفهم بنية منصة حجز الطيران YBF. نتمنى لك دوام التوفيق والنجاح في إدارة وتشغيل هذا المشروع الرائد والتميز! تم بحمد الله إعداد وتوثيق هذا الدليل الفني